



Extending a High-Performance Prover to Higher-Order Logic

Petar Vukmirović¹, Jasmin Blanchette^{1,2}, and Stephan Schulz³

¹ Vrije Universiteit Amsterdam, Amsterdam, The Netherlands
`petar.vukmirovic2@gmail.com, j.c.blanchette@vu.nl`

² Université de Lorraine, CNRS, Inria, LORIA, Nancy, France

³ DHBW Stuttgart, Stuttgart, Germany
`stephan.schulz@dhbw-stuttgart.de`

Abstract. Most users of proof assistants want more proof automation. Some proof assistants discharge goals by translating them to first-order logic and invoking an efficient prover on them, but much is lost in translation. Instead, we propose to extend first-order provers with native support for higher-order features. Building on our extension of E to λ -free higher-order logic, we extend E to full higher-order logic. The result is the strongest prover on benchmarks exported from a proof assistant.

1 Introduction

In the last few decades, proof assistants have become indispensable tools for developing trustworthy formal proofs. They are used both in academia to verify mathematical theories [17] and in industry to verify the correctness of hardware [21] and software [16,22,24]. However, due to the lack of strong built-in proof automation, proving seemingly simple goals can be a tedious manual task. To mitigate this, many proof assistants include a subsystem such as CoqHammer, HOL(y)Hammer, or Sledgehammer [9] that translates higher-order goals to first-order logic and passes them to efficient first-order automatic provers. If a first-order prover succeeds, the proof is reconstructed and the goal is closed.

Unfortunately, the translation of higher-order constructs is clumsy and leads to poor performance on goals that require higher-order reasoning. Using native higher-order provers such as Satallax [10] as backends is not always a good solution because they are much less efficient than their first-order counterparts [37]. To bridge this gap, in 2016 we proposed to develop a new generation of higher-order provers that extend the arguably most successful first-order calculus, superposition, to higher-order logic, starting from a position of strength.

Our research has focused on three milestones: supporting λ -free higher-order logic, adding λ -terms, and adding first-class Boolean terms. In 2019, we extended the state-of-the-art first-order prover E [32] with a λ -free superposition calculus [42], obtaining a version of E called Ehoh, as a stepping stone towards full higher-order logic. Together with Bentkamp, Tourret, and Waldmann, we have since developed calculi, called λ -superposition, corresponding to the other two milestones [5,4] and implemented them in the experimental superposition prover

Zipperposition [14]. This OCaml prover is not nearly as efficient as E. Nevertheless, it has won the higher-order division of the CASC prover competition [39] in 2020, 2021, and 2022, ending nearly a decade of Satallax domination.

We now fulfill a four-year-old promise: We present the extension of Ehoh to full higher-order logic (Sect. 2) based on incomplete variants of λ -superposition. We call this prover λ E. In λ E's implementation, we used the extensive experience with Zipperposition to choose a set of effective rules that could easily be retrofitted into an originally first-order prover. Another guiding principle was *gracefulness*: Our changes should not impact the strong first-order performance of E and Ehoh.

One of the main challenges we faced was retrofitting λ -terms in Ehoh's term representation (Sect. 3). Furthermore, Ehoh's inference engine assumes that inferences compute a most general unifier. We implemented a higher-order unification procedure [41] that can return multiple unifiers (Sect. 4) and integrated it in the inference engine. Finally, we extended and adapted the superposition rule, resulting in an incomplete, pragmatic variant of λ -superposition (Sect. 5).

We evaluated λ E on a selection of proof assistants benchmarks as well as all higher-order theorems in the TPTP library [38] (Sect. 6). λ E outperformed all other higher-order provers on the proof assistant benchmarks; on the TPTP benchmarks, it ended up second only to the cooperative version of Zipperposition, which employs Ehoh as a backend. An arguably fairer comparison without the backend puts λ E in first place for both benchmark suites. We also compared the performance of λ E with E on first-order problems and found that no overhead has been introduced by the extension to higher-order logic.

λ E is part of the E prover's development repository and will be part of E 3.0. It can be enabled by passing the option `--enable-ho` to the `configure` script. E and λ E's source code is freely available online.¹

2 Logic

Our target logic is monomorphic classical higher-order logic with Hilbert choice. The following text is partly based on Vukmirović et al. [40, Sect. 2].

Terms s, t, u, v are inductively defined as free variables $F, X, \dots$, bound variables $x, y, z, \dots$, constants $f, g, a, b, \dots$, applications st , and λ -abstractions $\lambda x. s$. Bound variables may be *loose* (e.g., y in $\lambda x. ya$) [27].

We let $s\bar{t}_n$ stand for $s t_1 \dots t_n$ and $\lambda\bar{x}_n. s$ for $\lambda x_1 \dots \lambda x_n. s$. Every β -normal term can be written as $\lambda\bar{x}_m. s\bar{t}_n$, where s is not an application; we call s the *head* of the term. If s is a free variable, we call the term *flex*; otherwise, the term is *rigid*. A term of type o , where o is the distinguished Boolean type, is called a *formula*. A term whose type is of the form $\tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow o$ is called a *predicate*. Logical symbols are part of the signature and may thus occur within terms. We write them in bold: $\perp, \top, \neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists, \approx$.

On top of the terms, we define some clausal structure. This structure is needed by λ -superposition. A literal l is an equation $s \approx t$ or a disequation $s \not\approx t$. A clause is a finite multiset of literals, interpreted and written disjunctively: $l_1 \vee \dots \vee l_n$.

¹ <https://github.com/eprover/eprover.git>

3 Terms

E is designed around perfect term sharing [25], a principle that we kept in Ehoh and λE : Any two structurally identical terms are guaranteed to be the same object in memory. This is achieved through term *cells*, which represent individual terms. Each cell has (among other fields) (1) `f_code`, an integer corresponding to the symbol at the head of the term (negative if the head is a free variable, positive otherwise); (2) `num_args`, corresponding to the number of arguments applied to the head; and (3) `args`, an array of size `num_args` of pointers to argument terms. We use the notation $f(s_1, \dots, s_n)$ to denote a cell whose `f_code` corresponds to `f`, `num_args` equals n , and `args` points to the cells for $s_1, \dots, s_n$.

Like Leo-III [33, Sect. 4.8], Ehoh represents λ -free higher-order terms using a flattened, spine notation [12]. Thus, the terms `f`, `f a`, and `f a b` are represented by the cells `f`, `f(a)`, and `f(a, b)`. To ensure that free variables are perfectly shared, Ehoh treats applied free variables differently: Arguments are not applied directly to a free variable, but using a distinguished symbol `@` of variable arity. For example, the term $X a b$ is represented by the cell `@(X, a, b)`. This ensures that two different occurrences of the free variable X correspond to the same object, which makes substitutions more efficient [42].

Representation of λ -Terms. To support full higher-order logic, Ehoh's λ -free cell data structure must be extended to support the λ binder. We use the locally nameless representation [13]: De Bruijn indices represent (possibly loose) bound variables, whereas we keep the current representation for free variables.

Extending the term representation of Ehoh with a new term kind involves intricate manipulation of the cell data structure. De Bruijn indices must be represented like other cells with either a negative or a positive `f_code`, but the code must clearly identify that the cell is a De Bruijn index.

Apart from during β -reduction, De Bruijn indices mostly behave like constants. Therefore, we choose to represent De Bruijn indices using positive `f_codes`: The De Bruijn index i will have `f_code i`. To ensure that De Bruijn indices are not mistaken for function symbols, we use the cell's `properties` bitfield, which holds precomputed properties. We introduce the property `IsDBVar` to denote that the cell represents a De Bruijn index. De Bruijn indices are systematically created through a dedicated function that sets the `IsDBVar` property. When given the same De Bruijn index and type, this function always returns the same object. Finally, we guard all the functions and macros that manipulate function codes to check if the property `IsDBVar` is set. To ensure perfect sharing of De Bruijn indices, arguments to De Bruijn indices are applied like for free variables, using `@`.

Extending cells to support λ -abstraction is easier. Each λ -abstraction has the distinguished function code `LAM` as the head symbol and two arguments: (1) a De Bruijn index 0 of the type of the abstracted variable; (2) the body of the λ -abstraction. Consider the term $\lambda x. \lambda y. f x x$, where both x and y have the type ι . This term is represented as `$\lambda \lambda f 1 1$` in locally nameless representation, where bold numbers represent De Bruijn indices. In λE , the same term is represented by the cell `LAM(0, LAM(0, f(1, 1)))`, where all De Bruijn variables have type ι .

The first argument of **LAM** is redundant, since it can be deduced from the type of the λ -abstraction. However, basic λ -term manipulation operations often require access to this term. We store it explicitly to avoid creating it repeatedly.

Efficient β -Reduction. Terms are stored in $\beta\eta$ -reduced form. As these two reductions are performed very often, they ought to be efficient. Ehoh performs β -reduction by reducing the leftmost outermost β -redex first. To represent β -redexes, E uses the $\textcircled{\text{e}}$ symbol. Thus, the term $(\lambda x. \lambda y. (x y)) f a$ is represented by $\textcircled{\text{e}}(\text{LAM}(\mathbf{0}, \text{LAM}(\mathbf{0}, \textcircled{\text{e}}(\mathbf{1}, \mathbf{0}))), f, a)$. Another option would have been to add arguments applied to λ -terms directly to the λ representation (as in $\text{LAM}(\mathbf{0}, \text{LAM}(\mathbf{0}, \textcircled{\text{e}}(\mathbf{1}, \mathbf{0})), f, a)$), but this would break the invariant that **LAM** has two arguments. Furthermore, replacing free variables with λ -abstractions (e.g., replacing X with $\lambda x. x$ in $\textcircled{\text{e}}(X, a)$) would require additional normalization.

A term can be β -reduced as follows: When a cell $\textcircled{\text{e}}(\text{LAM}(\mathbf{0}, s), t)$ is encountered, the field **binding** (normally used to record the substitution for a free variable) of the cell $\mathbf{0}$ is set to t . Then s is traversed to instantiate every loose occurrence of $\mathbf{0}$ in s with **binding**, whose loose De Bruijn indices are shifted by the number of λ binders above the occurrence of $\mathbf{0}$ in s [20]. Next, this procedure is applied to the resulting term and its subterms, in leftmost outermost fashion.

λE 's β -normalization works in this way, but it features a few optimizations. First, given a term of the form $(\lambda \bar{x}_n. s) \bar{t}_n$, λE , like Leo-III [34], replaces the bound variables x_i with t_i in parallel. Avoiding the construction of intermediate terms reduces the number of recursive function calls and calls to the cell allocator.

Second, in line with the gracefulness principle, we want λE to incur little (or no) overhead on first-order problems and to excel on higher-order problems with a large first-order component. If β -reduction is implemented naively, finding a β -redex involves traversing the entire term. On purely first-order terms, β -reduction is then a waste of time. To avoid this, we use Ehoh's perfectly shared terms and their **properties** field. We introduce the property **HasBetaReducibleSubterm**, which is set if a cell is β -reducible. Whenever a new cell that contains a β -reducible term as a direct subterm is shared, the property is set. Setting of the property is inductively continued when further superterms are shared. For example, in the term $t = f a (g((\lambda x. x) a))$, the cells for $(\lambda x. x) a$, $g((\lambda x. x) a)$, and t itself have the property **HasBetaReducibleSubterm** set. When it needs to find β -reducible subterms, λE will visit only the cells with this property set. This further means that on first-order subterms, a single bit masking operation is enough to determine that no subterm should be visited.

Along similar lines, we introduce a property **HasDBSubterm** that caches whether the cell contains a De Bruijn subterm. This makes instantiating De Bruijn indices during β -normalization faster, since only the subterms that contain De Bruijn indices must be visited. Similarly, some other operations such as shifting De Bruijn indices or determining whether a term is closed (i.e., it contains no loose bound variables) can be sped up or even avoided if the term is first-order.

Efficient η -Reduction. The term $\lambda x. s x$ is η -reduced to s whenever x does not occur unbound in s . Observing that a term cannot be η -reduced if it contains

no λ -abstractions, we introduce a property `HasLambda` that notes the presence of λ 's in a term. Only terms with λ 's are visited during η -reduction.

λ E performs parallel η -reduction: It recognizes terms of the form $\lambda \bar{x}_m. s \bar{x}_m$ such that none of the x_i occurs unbound in s . If done naively, reducing terms of this kind requires up to m traversals of s to check if each x_i occurs in s . In λ E, exactly one traversal of s is required. More precisely, when η -reducing a cell $\text{LAM}(\mathbf{0}, s)$, λ E considers all λ binders in s as well. In general, the cell will be of the form $\text{LAM}(\mathbf{0}, \dots, \text{LAM}(\mathbf{0}, t) \dots)$, where t is not a λ -abstraction, and l is the number of `LAM` symbols above t . Then λ E breaks the body t down into a decomposition $u(\mathbf{n} - \mathbf{1}) \dots \mathbf{1} \mathbf{0}$ where u is not of the form $\dots \mathbf{n}$; such a decomposition is unique. If $n = 0$, the cell is not η -reducible. Otherwise, u is traversed to determine the minimal index j of a loose De Bruijn index, taking $j = \infty$ if no such index exists. λ E can then remove the $k = \min\{j, l, n\}$ rightmost outermost λ binders in $\text{LAM}(\mathbf{0}, \dots, \text{LAM}(\mathbf{0}, t) \dots)$ and replace t by the variant of $u(\mathbf{n} - \mathbf{1}) \dots (\mathbf{k} + \mathbf{1}) \mathbf{k}$ obtained by shifting the loose De Bruijn indices down by k .

To illustrate this convoluted De Bruijn arithmetic, we consider the term $\lambda x. \lambda y. \lambda z. f x y z$. This term is represented by the cell $\text{LAM}(\mathbf{0}, \text{LAM}(\mathbf{0}, \text{LAM}(\mathbf{0}, f(\mathbf{2}, \mathbf{2}, \mathbf{1}, \mathbf{0}))))$. λ E splits $f(\mathbf{2}, \mathbf{2}, \mathbf{1}, \mathbf{0})$ into two parts: $u = f \mathbf{2}$ and the arguments $\mathbf{2}, \mathbf{1}, \mathbf{0}$. Since the minimal index in u is $\mathbf{2}$, we can omit the De Bruijn indices $\mathbf{1}$ and $\mathbf{0}$ and their λ binders, yielding the η -reduced cell $\text{LAM}(\mathbf{0}, f(\mathbf{0}, \mathbf{0}))$.

Parallel η -reduction both speeds up η -reduction and avoids creating intermediate terms. For finding the minimal loose De Bruijn index, optimizations such as the `HasDBSubterm` property are used.

Representation of Boolean Terms. `E` and `Ehoh` represent Boolean terms using cells whose `f_codes` are reserved for logical symbols. Quantified formulas are represented by cells in which the first argument is the quantified variable and the second one is the body of the quantified formula. For example, the term $\forall x. p x$ corresponds to the cell $\forall(X, p(X))$, where X is a free variable. This representation is convenient for parsing and clausification, which is what `E` and `Ehoh` use it for, but in full higher-order logic, it is problematic during proof search: Booleans can occur as subterms in clauses, as in $q(X) \vee p(\forall(X, r(X)))$, and instantiating X in the first literal should not affect X in the second literal.

To avoid this issue, in λ E we use λ binders to represent quantified formulas, as is customary in higher-order logic [1, §51]. Thus, $\forall x. s$ is represented by $\forall(\lambda x. s)$. Quantifiers are then unary symbols that do not directly bind the variables. Since λ E represents bound variables using De Bruijn indices, this solves all α -conversion issues. However, this solution is incompatible with thousands of decades-old lines of clausification code that assumes `E`'s representation of quantifiers. Therefore, λ E converts quantified formulas only after clausification, for Boolean terms that occur in a higher-order context (e.g., as argument to a function symbol).

New Term Orders. The λ -superposition calculus is parameterized by a term order that is used to break symmetries in the search space. We implemented the versions of the Knuth–Bendix order (KBO) and lexicographic path order (LPO) for higher-order terms described by Bentkamp et al. [4]. These orders encode

λ -terms as first-order terms and then invoke the standard KBO or LPO. For efficiency, we implemented separate KBO and LPO functions that compute the order directly, intertwining the encoding and the order computation.

Ehoh cells contain a **binding** field that can be used to store the substitution for a free variable. Substitutions can then be applied by following the **binding** pointers, replacing each free variable with its instance. Thus, when Ehoh needs to perform a KBO or LPO comparison of an instantiated term, it needs only follow the **binding** pointers. In full higher-order logic, however, instantiating a variable can trigger a chain of $\beta\eta$ -reductions, changing the shape of the term dramatically. To prevent this, λE computes the $\beta\eta$ -reduced instances of the terms before comparing them using KBO or LPO.

4 Unification, Matching, and Term Indexing

Standard superposition crucially depends on the concept of a most general unifier (MGU). In higher-order logic, the concept is replaced by that of a complete set of unifiers (CSU), which may be infinite. Vukmirović et al. [41] designed an efficient procedure to enumerate a CSU for a term pair. It is implemented in Zipperposition, together with some extensions to term indexing. In λE , we further improve the performance of this procedure by implementing a terminating, incomplete variant. We also introduce a new indexing data structure.

The Unification Procedure. The unification procedure works by maintaining a list of unification pairs to be solved. After choosing a pair, it first normalizes it by β -reducing and instantiating the heads of both terms in the pair. Then, if either head is a variable, it computes an appropriate binding for this variable, thereby approximating the solution.

Unlike in first-order and λ -free higher-order unification, in the full higher-order case there may be many bindings that lead to a solution. To reduce this mostly blind guessing of bindings, the procedure features support for *oracles* [41]. These are procedures that solve the unification problem for a subclass of higher-order terms on which unification is decidable and, for λE , unary. Oracles help increase performance, avoid nontermination, and avoid redundant bindings.

Vukmirović et al. described their procedure as a transition system. In λE , the procedure is implemented nonrecursively, and the unifiers are enumerated using an iterator object that encapsulates the state of the unifier search. The iterator consists of five fields: (1) *constraints*, which holds the unification constraints; (2) *bt_state*, a stack that contains information necessary to backtrack to a previous state; (3) *branch_iter*, which stores how far we are in exploring different possibilities from the current search node; (4) *steps*, which remembers how many different unification bindings (such as imitation, projection, and identification) are applied; and (5) *subst*, a stack storing the variables bound so far.

The iterator is initialized to hold the original problem in *constraints*, and all other fields are initially empty. The unifiers are retrieved one by one by calling the function `FORWARDITER`. It returns `TRUE` if the iterator made progress, in

which case the unifier can be read via the iterator's *subst* field. Otherwise, no more unifiers can be found, and the iterator is no longer valid. The function's pseudocode is given below, including two auxiliary functions:

```

function NORMALIZEHEAD(t) is
  if t.head = @ ∧ t.args[0].is_lambda() then
    reduce the top-level  $\beta$ -redex in t
    return NORMALIZEHEAD(t)
  else if t.head.is_var() ∧ t.head.binding ≠ NIL then
    t.head ← t.head.binding
    return NORMALIZEHEAD(t)
  else
    return t

function BACKTRACKITER(iter) is
  if iter.bt_state.empty() then
    clear all fields in iter
    return FALSE
  else
    pop (constraints, branch_iter, steps, subst) from iter.bt_state
    set the corresponding fields of iter
    return TRUE

function FORWARDITER(iter) is
  forward ← ¬ iter.constraints.empty() ∨ BACKTRACKITER(iter)
  while forward ∧ ¬ iter.constraints.empty() do
    (lhs, rhs) ← pop pair from iter.constraints
    lhs ← NORMALIZEHEAD(lhs)
    rhs ← NORMALIZEHEAD(rhs)
    normalize and discard the  $\lambda$  prefixes of lhs and rhs
  if ¬lhs.head.is_var() ∧ rhs.head.is_var() then
    swap lhs and rhs
  if lhs.head.is_var() then
    oracle_res ← FIXPOINT(lhs, rhs, iter.subst)
  if oracle_res = NOTINFRAGMENT then
    oracle_res ← PATTERN(lhs, rhs, iter.subst)
  if oracle_res = NOTUNIFIABLE then
    forward ← BACKTRACKITER(ITER)
  else if oracle_res = NOTINFRAGMENT then
    n_steps, n_branch_iter, n_binding ←
      NEXTBINDING(lhs, rhs, iter.steps, iter.branch_iter)
  if n_branch_iter ≠ BINDEND then
    push pair (lhs, rhs) back to iter.constraints
    push quadruple (iter.constraints, n_branch_iter,
      iter.steps, iter.subst) onto iter.bt_state
    extend iter.subst with n_binding

```

```

    iter.steps ← n_steps
    iter.branch_iter ← BINDBEGIN
    else if lhs.head = rhs.head then
        create constraint pairs of arguments of lhs and rhs
        and push them to iter.constraints
        iter.branch_iter ← BINDBEGIN
    else if lhs.head = rhs.head then
        create constraint pairs of arguments of lhs and rhs
        and push them to iter.constraints
    else
        forward ← BACKTRACKITER(iter)
    return forward

```

FORWARDITER begins by backtracking if the previous attempt was successful (i.e., all constraints were solved). If it finds a state from which it can continue, it takes term pairs from *constraints* until there are no more constraints or it is determined that no unifier exists. The terms are normalized by instantiating the head variable with its binding and reducing the potential top-level β -redex that might appear. This instantiation and reduction process is repeated until there are no more top-level β -redexes and the head is not a variable bound to some term. Then the term with shorter λ prefix is expanded (only on the top level) so that both λ prefixes have the same length. Finally, the λ prefix is ignored, and we focus only on the body. In this way, we avoid fully substituting and normalizing terms and perform just enough operations to determine the next step of the procedure.

If either term of the constraint is flex, we first invoke oracles to solve the constraint. λ E implements the most efficient oracles implemented in Zipperposition: fixpoint and pattern [41, Sect. 6]. An oracle can return three results: (1) there is an MGU for the pair (UNIFIABLE), which is recorded in *subst*, and the next pair in *constraints* is tried; (2) no MGU exists for the pair (NOTUNIFIABLE), which causes the iterator to backtrack; (3) if the pairs do not belong to the subclass that oracle can solve (NOTINFRACTION), we generate possible variable bindings—that is, we guess the approximate form of the solution.

λ E has a dedicated module that generates bindings (NEXTBINDING). This module is given the current constraint and the values of *branch_iter* and *steps*, and it either returns the next binding and the new values of *branch_iter* and *steps* or reports that all different variable bindings are exhausted. The bindings that λ E's unification procedure creates are imitation, Huet-style projection, identification, and elimination (one argument at a time) [41, Sect. 3]. A limit on the total number of applied binding rules can be set, as well as a limit on the number of individual rule applications. The binding module checks whether limits are reached using the iterator's *steps* field.

Computing bindings is the only point in the procedure where the search tree branches and different possibilities are explored. Thus, when λ E follows the branch indicated by the binding module, it records the state to which it needs to return should the followed branch be backtracked. The state consists of the values of *constraints*, *steps*, and *subst* before the branch is followed and the value

of *branch_iter* that points past the followed branch. The values of *branch_iter* are either `BINDBEGIN`, which denotes that no binding was created, intermediate values that `NEXTBINDING` uses to remember how far through bindings it is, and `BINDEND`, which indicates that all bindings are exhausted.

If all bindings are exhausted, the procedure checks whether the pair is flex–flex and both sides have the same head. If so, the pair is decomposed and constraints are derived from the pair’s arguments; otherwise, the iterator backtracks. If the pair is rigid–rigid, for unification to succeed, the heads of both sides must be the same. Unification then continues with new constraints derived from the arguments. Otherwise, the iterator must be backtracked.

Matching. In E, the matching algorithm is mostly used inside simplification rules such as demodulation and subsumption [29]. As these rules must be efficiently performed, using a complex matching algorithm is not viable. Instead, we provide a matching algorithm for the pattern class of terms [27] to complement Ehoh’s λ -free higher-order matching algorithm [42, Sect. 4]. A term is a *pattern* if each of its free variables either has no arguments (as in first-order logic) or is applied to distinct De Bruijn indices.

To help determine whether to use the pattern or λ -free algorithm, we introduce a cached property `HasNonPatternVar`, which is set for terms of the form $X \bar{s}_n$ where $n > 0$ and either there exists some s_i that is not a De Bruijn index or there exist indices $i < j$ such that $s_i = s_j$ is a De Bruijn index. This property is propagated to the superterms when they are perfectly shared. This allows later checks if a term belongs to the pattern class to be performed in constant time.

We modify the λ -free higher-order matching algorithm to treat λ prefixes as above in the unification procedure—by bringing the prefixes to the same length and ignoring them afterwards. This ensures that the algorithm will never try to match a free variable with a λ -abstraction, making sure that β -redexes never appear. We also modify the algorithm to ensure that free variables are never bound to terms that have loose bound variables. This algorithm cannot find many complex matching substitutions (matchers), but it can efficiently determine whether two terms are variable renamings of each other or whether a simple matcher can be used, as in the case of $(X (\lambda x. x) \mathbf{b}, \mathbf{f} (\lambda x. x) \mathbf{b})$, where $X \mapsto \mathbf{f}$ is usually the desired matcher. If this algorithm does not find a matcher and both terms are patterns, pattern matching is tried.

Indexing. E, like other modern theorem provers, efficiently retrieves unifiable or matchable pairs of terms using indexing data structures. To find terms unifiable with a query term or instances of a query term, it uses *fingerprint indexing* [30]. Vukmirović et al. extended this data structure to support full higher-order terms in Zipperposition [41, Sect. 6]. We use the same approach in λ E, and we extend feature vector indices [31] in the same way.

E uses *perfect discrimination trees* [26] to find generalizations of the query term (i.e., terms of which the query term is an instance). This data structure is a trie that indexes terms by representing them in a serialized, flattened form. The left branch from the root in Figure 1 shows how the first-order terms $\mathbf{f} \mathbf{a} X$

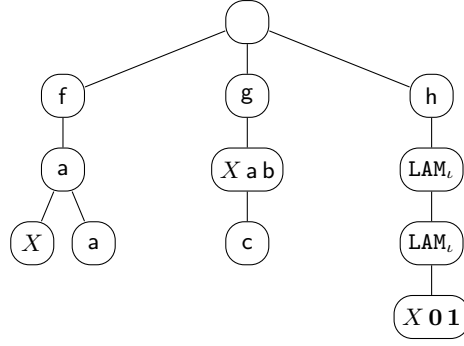


Fig. 1. First-order, λ -free higher-order, and higher-order pattern terms in a perfect discrimination tree

and $f\ a\ a$ are stored. In Ehoh, this data structure is extended to support partial application and applied variables [42].

In λE , we extend this structure to support λ -abstractions and the higher-order pattern matching algorithm. To this end, we change the way in which terms are serialized. First, we require that all terms are fully η -expanded (except for arguments of variables applied in patterns). Then, when the term is serialized, we use a single node for applied variable terms $X\ \bar{s}_n$, instead of a node for X followed by nodes for the arguments $\bar{s}_n$. We serialize the λ -abstraction $\lambda x. s$ using a dedicated node LAM_τ , where τ is the type of x , followed by the serialization of s . Other than these changes, serialization remains as in Ehoh, following the gracefulness principle. Figure 1 shows how $g\ (X\ a\ b)\ c$ and $h\ (\lambda x. \lambda y. X\ y\ x)$ are serialized. Since the terms are stored in serialized form, it is hard to manipulate λ prefixes of stored terms during matching. Performing η -expansion when serializing terms ensures that matchable terms have λ prefixes of the same length.

We have dedicated separate nodes for applied variables because access to arguments of applied variables is necessary for the pattern matching algorithm. Even though arguments can be obtained by querying the arity n of the variable and taking the next n arguments in the serialization, this is both inefficient and inelegant. As for De Bruijn indices, we treat them the same as function symbols.

Following the notation from the extension of perfect discrimination trees to λ -free higher-order logic [42], we now describe how enumeration of generalizations is performed. To traverse the tree, λE begins at the root node and maintains two stacks: **term_stack** and **term_proc**, where **term_stack** contains the subterms of the query term that have to be matched, and **term_proc** contains processed terms that are used to backtrack to previous states. Initially, **term_stack** contains the query term, the current matching substitution σ is empty, and the successor node is chosen among the child nodes as follows:

- A. If the node is labeled with a symbol ξ (where ξ is either a De Bruijn index or a constant) and the top item t of **term_stack** is of the form $\xi\ \bar{t}_n$, replace t by n new items $t_1, \dots, t_n$, and push t onto **term_proc**.

- B. If the node is labeled with a symbol LAM_τ and the top item t of **term_stack** is of the form $\lambda x. s$ and the type of x is τ , replace t by s , and push t onto **term_proc**.
- C. If the node is labeled with a possibly applied variable $X \bar{s}_n$ (where $n \geq 0$), and the top item of **term_stack** is t , the matching algorithm described above is run on $X \bar{s}_n$ and t . The algorithm takes into account σ built so far and extends it if necessary. If the algorithm succeeds, pop t from **term_stack**, push it onto **term_proc**, and save the original value of σ in the node.

Backtracking works in the opposite direction: If the current node is labeled with a De Bruijn index or function symbol node of arity n , pop n terms from **term_stack** and move the top of **term_proc** to **term_stack**. If the node is labeled with LAM_τ , pop the top of **term_stack** and move the top of **term_proc** to **term_stack**. Finally, if the node is labeled with a possibly applied variable, move the top of the **term_proc** to **term_stack** and restore the value of σ .

As an example of how finding a generalization works, when looking for generalizations of $g(f a b) c$ in the tree of Figure 1, the following states of stacks and substitutions emerge, from left to right:

	ϵ	g	$g.(X a b)$	$g.(X a b).c$
term_stack	$[g(f a b) c]$	$[f a b, c]$	$[c]$	$[]$
term_proc	$[]$	$[g(f a b) c]$	$[f a b, g(f a b) c]$	$[c, f a b, g(f a b) c]$
σ	$\emptyset$	$\emptyset$	$\{X \mapsto f\}$	$\{X \mapsto f\}$

5 Preprocessing, Calculus, and Extensions

Ehoh’s simple λ -free higher-order calculus performed well on Sledgehammer problems and formed a promising stepping stone to full higher-order logic [42]. When implementing support for full higher-order logic, we were guided by efficiency and gracefulness with respect to Ehoh’s calculus rather than completeness. Whereas Zipperposition provides both complete and incomplete modes, λE only offers incomplete modes.

Preprocessing. Our experience with Zipperposition showed the importance of flexibility in preprocessing the higher-order problems [40]. Therefore, we implemented a flexible preprocessing module in λE .

To maintain compatibility with Ehoh, λE can optionally transform all λ -abstractions into named functions. This process is called *λ -lifting* [19]. λE also removes all occurrences of Boolean subterms (other than $\perp$, $\top$, and free variables) in higher-order contexts using a FOOL-like transformation [23]. For example, the formula $f(p \wedge q) \approx a$ becomes $(p \wedge q \rightarrow f(\top) \approx a) \wedge (\neg(p \wedge q) \rightarrow f(\perp) \approx a)$.

Many TPTP problems use the **definition** role to identify the definitions of symbols. λE can treat definition axioms as rewrite rules, and replace all occurrences of defined symbols during preprocessing. Furthermore, during SInE [18] axiom selection, it can always include the defined symbol in the trigger relation.

Calculus. λE implements the same superposition calculus as Ehoh with three important changes. First, wherever Ehoh requires the MGU of terms, λE enumerates unifiers from a finite subset of the CSU, as explained in Sect. 4. Second, λE uses versions of the KBO and LPO orders designed for λ -terms.

The third difference is more subtle. One of the main features of Ehoh is *prefix optimization* [42, Sect. 1]: a method that, given a demodulator $s \approx t$, makes it possible to replace both applied and unapplied occurrences of s by t by traversing only the first-order subterms of a rewritable term. In a λ -free setting, this optimization is useful, but in the presence of $\beta\eta$ -normalization, the shapes of terms can change drastically, making it much harder to track prefixes of terms. This is why we disable the prefix optimization in λE . To compensate for losing this optimization, we introduce the argument congruence rule AC in λE and enable positive and negative functional extensionality (PE and NE) by default:

$$\frac{s \approx t \vee C}{s X \approx t X \vee C} \text{AC} \quad \frac{s \not\approx t \vee C}{s (\text{sk } \overline{X}) \not\approx t (\text{sk } \overline{X}) \vee C} \text{NE} \quad \frac{s X \approx t X \vee C}{s \approx t \vee C} \text{PE}$$

AC and NE assume that s and t are of function type. In NE, $\overline{X}$ denotes all the free variables occurring in s and t , and sk is a fresh Skolem symbol of the appropriate type. PE has a side condition that X may not occur in s , t , or C .

Saturation. E 's saturation procedure assumes that each attempt to perform an inference will either result in a single clause or fail due to one of the inference side conditions. Unification procedures that produce multiple substitutions break this invariant, and the saturation procedure needed to be adjusted.

For Zipperposition, Vukmirović et al. developed a variant of the saturation procedure that interleaves computing unifiers and scheduling inferences to be performed [40]. Since completeness was not a design goal for λE , we did not implement this version of the saturation procedure. Instead, in places where previously a single unifier was expected, λE consumes all elements of the iterator used for enumerating a unifier, converting them into clauses.

Reasoning about Formulas. Even though most of the Boolean structure is removed during preprocessing, formulas can reappear at the top level of clauses during saturation. For example, after instantiating X with $\lambda x. \lambda y. x \wedge y$, the clause $X \text{ p q} \vee \text{ a} \approx \text{ b}$ becomes $(\text{ p} \wedge \text{ q}) \vee \text{ a} \approx \text{ b}$. λE converts every clause of the form $\varphi \vee C$, where φ has a logic symbol as its head, or it is a (dis)equation between two formulas different than $\top$, to an explicitly quantified formula. Then, the clausification algorithm is invoked on the formula to restore the clausal structure. Zipperposition features more dynamic clausification modes, but for simplicity we decided not to implement them in λE .

The λ -superposition calculus for full higher-order logic [4] includes many rules that act on Boolean subterms, which are necessary for completeness. Other than Boolean simplification rules, which use simple tautologies such as $\text{ p} \wedge \top \leftrightarrow \text{ p}$ to simplify terms, we have implemented none of the Boolean rules of this calculus in λE . First, we have observed that complicated rules such as FLUIDBOOLHOIST and

FLUIDLOOBHOIST are hardly ever useful in practice and usually only contribute to an uncontrolled increase in the proof state size. Second, simpler rules such as BOOLHOIST can usually be simulated by pragmatic rules that perform Boolean extensionality reasoning, described below.

To make up for excluding Boolean rules, we use an incomplete, but more easily controllable and intuitive rule, called *primitive instantiation*. This rule instantiates free predicate variables with approximations of formulas that are ground instances of this variable. We use the approximations described by Vukmirović and Nummelin [43, Sect. 3.3].

λE 's handling of the Hilbert choice operator is inspired by Leo-III's [35]. λE recognizes clauses of the form $\neg P X \vee P (f P)$, which essentially denote that f is a choice symbol. Then, when subterm $f s$ is found during saturation, s is used to instantiate the choice axiom for f . Similarly, Leibniz equality [43] is eliminated by recognizing clauses of the form $\neg P a \vee P b \vee C$. These clauses are then instantiated with $P \mapsto \lambda x. x \approx a$ and $P \mapsto \lambda x. x \not\approx b$, which results in $a \approx b \vee C$.

Finally, λE treats induction axioms specially. Like Zipperposition [40, Sect. 4], it abstracts literals from the goal clauses and instantiates induction axioms with these abstractions. Since Zipperposition supports dynamic calculus-level clausification, induction axioms are instantiated during saturation, when the axioms are processed. In λE , this instantiation is performed immediately after clausification. After λE has collected all the abstractions, it traverses the clauses and instantiates those that have applied variable of the same type as the abstraction.

Extensionality. λE takes a pragmatic approach to reasoning about functional and Boolean extensionality: It uses *abstracting* rules [5] which simulate basic superposition calculus rules but do not require unifiability of the partner terms in the inference. More precisely, assume a core inference needs to be performed between two β -reduced terms u and v , such that they can be represented as $u = C[s_1, \dots, s_n]$ and $v = C[t_1, \dots, t_n]$, where C is the most general “green” [5] common context of u and v , not all of s_i and t_j are free variables, and for at least one i , $s_i \neq t_i$, s_i and t_i are not possibly applied free variables, and they are of Boolean or function type. Then, the conclusion is formed by taking the conclusion D of the core inference rule (which would be created if s and t are unifiable) and adding literals $s_1 \not\approx t_1 \vee \dots \vee s_n \not\approx t_n$.

These rules are particularly useful because λE has no rules that dynamically process Booleans in FOOL-like fashion, such as BOOLHOIST. For example, given the clauses $f (p \wedge q) \approx a$ and $g (f p) \not\approx b$, the abstracting version of the superposition rule would result in $g a \not\approx b \vee (p \wedge q) \not\approx p$. In this way, the Boolean structure bubbles up to the top level and is further processed by clausification. We noticed that this alleviates the need for the other Boolean rules in practice.

6 Evaluation

We now try to answer two questions about λE : *How does λE compare against other higher-order provers (including Ehoh)? Does λE introduce any overhead*

compared with Ehoh? To answer these questions, we ran provers on problems from the TPTP library [38] and on benchmarks generated by Sledgehammer (SH) [28]. The experiments were carried out on StarExec Miami [36] nodes equipped with Intel Xeon E5-2620 v4 CPU clocked at 2.10 GHz. For the TPTP part, we used the CASC 2021² time limits: 120 s wall-clock and 960 s CPU. For SH benchmarks and to answer the other question, we used Sledgehammer’s default time limit: 30 s wall-clock and CPU. The raw evaluation data is available online.³

Comparison with Other Provers. To answer the first question, we let λE compete with the top contenders in the higher-order division of CASC 2021: *cvc5* 0.0.7 [2], *Ehoh* 2.7 [42], *Leo-III* 1.6.6 [35], *Vampire* 4.6 [8], and *Zipperposition* 2.1 [40]. We also included *Satallax* 3.5 [10]. We used all 2899 higher-order theorems in TPTP 7.5.0 as well as 5000 SH higher-order benchmarks originating from the Seventeen benchmark suite [15]. On SH benchmarks, *cvc5*, *Ehoh*, λE , *Vampire*, and *Zipperposition* were run using custom schedules provided by their developers, optimized for single-core usage and low timeouts. Otherwise, we used the corresponding CASC configurations.

Although it internally does not support λ -abstractions, *Ehoh* 2.7 can parse full higher-order logic using λ -lifting. We included two versions of *Zipperposition*: *coop* uses *Ehoh* 2.7 as a backend to finish proof attempts, whereas *uncoop* does not. Both *Ehoh* and λE were run in the automatic scheduling mode. Compared with *Ehoh*, λE features a redesigned module for automatic scheduling, it can exploit multiple CPU cores, and its heuristics have been more extensively trained on higher-order problems.

The results are shown in Figure 2. λE dramatically improves *E*’s higher-order reasoning capabilities compared with *Ehoh*. It solves 20% more TPTP benchmarks and 7% more SH benchmarks. The reason for the higher performance increase for TPTP is likely that TPTP benchmarks tend to require more higher-order reasoning than SH benchmarks, which often have a large first-order component and for which *Ehoh* was already very successful.

λE was envisioned as an efficient backend to proof assistants. As such, it excels on SH benchmarks, outperforming the competition. On TPTP, it outperforms all higher-order provers other than *Zipperposition-coop*. If *Zipperposition*’s *Ehoh* backend is disabled, λE outperforms *Zipperposition* by a wide margin. This comparison is arguably fairer; after all, λE does not use an older version of *Zipperposition* as a backend. These results suggest that λE already implements most of the necessary features for a high-performance higher-order prover but could benefit from the kind of fine-tuning that *Zipperposition* underwent in the last four years.

Remarkably, the raw evaluation data reveals that λE solves 181 SH problems and 24 TPTP problems that *Zipperposition-coop* does not. The lower number of uniquely solved TPTP problems is likely because *Zipperposition* was heavily optimized on the TPTP.

² <http://www.tptp.org/CASC/28/>

³ <https://doi.org/10.5281/zenodo.6389849>

	TPTP	SH
cvc5	1931	2577
Ehoh	2105	2611
λ E	2533	2804
Leo-III	2282	1601
Satallax	2320	1719
Vampire	2203	2240
Zipperposition-coop	2583	2754
Zipperposition-uncoop	2483	2181

Fig. 2. Comparison of higher-order provers

	TPTP
Ehoh FO	535
Ehoh HO	538
λ E FO	537
λ E HO	541

Fig. 3. Evaluation of λ E’s overhead

Comparison with the First-Order E. Both Ehoh and λ E can be compiled in a mode that disables most of the higher-order reasoning. This mode is designed for users that are interested only in E’s first-order capabilities and care a lot about performance. To answer the second evaluation question, about assessing overhead of λ E, we chose all the 1138 unique problems used at CASC from 2019 to 2021 in the first-order theorem division and ran Ehoh and λ E both in this first-order (FO) mode and in higher-order (HO) mode.

We fixed a single configuration of options, because Ehoh’s and λ E’s automatic scheduling methods could select different configurations and we would not be measuring the overhead but the quality of the chosen configurations. We chose the *boa* configuration [42, Sect. 7], which is the configuration most often used by E 2.2 in its automatic scheduling mode. The results are shown in Figure 3.

Counterintuitively, the higher-order versions of both provers outperform the first-order counterparts. However, the difference is so small that it can be attributed to the changes to memory layout that affect the order in which clauses are chosen. Similar effects are visible when comparing the first-order versions.

CASC Results. λ E also took part in CASC 2022. In the TPTP higher-order division, λ E finished second, after Zipperposition, as expected. In the Sledgehammer division, λ E tied with Ehoh for first place, a disappointment. The likely explanation is that λ E used a wrong configuration in this division, as we found out afterwards. We expect better performance at CASC 2023.

7 Discussion and Related Work

On the trajectory to λ E, we developed, together with colleagues, three superposition calculi: for λ -free higher-order logic [6], for a higher-order logic with λ -abstraction but no Booleans [5], and for full higher-order logic [5]. These milestones allowed us to carefully estimate how the increased reasoning capabilities of each calculus influence its performance.

Extending first-order provers with higher-order reasoning capabilities has been attempted by other researchers as well. Barbosa et al. extended the SMT

solvers CVC4 (now `cvc5`) and `veriT` to higher-order logic in an incomplete way [3]. Bhayat and Reger first extended Vampire to higher-order logic using combinatory unification [8], an incomplete approach, before they designed and implemented a complete higher-order superposition calculus based on SKBCI combinators [7]. The advantage is that combinators can be supported as a thin layer on top of λ -free terms. This calculus is also implemented in Zipperposition. However, in informal experiments, we found that λ -superposition performs substantially better, corroborating the CASC results, so we decided to make a more profound change to Ehoh and implement λ -superposition.

Possibly the only actively maintained higher-order provers built from the bottom up as higher-order provers are Leo-III [35] and Satallax’s [10] successor Lash [11]. A further overview of other traditional higher-order provers and the calculi they are based on can be found in the paper about Ehoh [42, Sect. 9].

8 Conclusion

In 2019, the reviewers of our Ehoh paper [42] were skeptical that extending Ehoh with support for full higher-order logic would be feasible. One of them wrote:

A potential criticism could be that this step from E to Ehoh is just extending FOL by those aspects of HOL that are easily in reach with rather straightforward extensions (none of the extensions is indeed very complicated), and that the difficult challenges of fully supporting HOL have yet to be confronted.

We ended up addressing the theoretical “difficult challenges” in other work with colleagues. In this paper, we faced the practical challenges pertaining to the extension of Ehoh’s data structures and algorithms to support full higher-order logic and demonstrated that such an extension is possible. Our evaluation shows that this extension makes λ E the best higher-order prover on benchmarks coming from interactive theorem proving practice, which was our goal. λ E lags slightly behind Zipperposition on TPTP problems. One reason might be that Zipperposition does not assume a clausal structure and can perform subtle formula-level inferences. It would be useful to implement the same features in λ E. We have also only started tuning λ E’s heuristics on higher-order problems.

Acknowledgment. Ahmed Bhayat and Martin Suda provided Vampire configurations optimized for Sledgehammer. Andrew Reynolds did the same for `cvc5`. Jannis Limperg helped us debug the submission artifact. Simon Cruanes, Wan Fokink, Mark Summerfield, and the anonymous reviewers suggested several textual improvements. We thank them all.

This research has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (grant agreement No. 713999, Matryoshka). Vukmirović and Blanchette have received funding from the Netherlands Organization for Scientific Research (NWO) under the Vidi program (project No. 016.Vidi.189.037, Lean Forward).

References

1. Andrews, P.B.: An Introduction to Mathematical Logic and Type Theory: To Truth Through Proof (2nd Ed.), Applied Logic, vol. 27. Springer (2002)
2. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: *cvc5: A versatile and industrial-strength SMT solver*. In: Fisman, D., Rosu, G. (eds.) TACAS 2022. LNCS, vol. 13243, pp. 415–442. Springer (2022)
3. Barbosa, H., Reynolds, A., El Ouraoui, D., Tinelli, C., Barrett, C.W.: Extending SMT solvers to higher-order logic. In: CADE. LNCS, vol. 11716, pp. 35–54. Springer (2019)
4. Bentkamp, A., Blanchette, J., Tourret, S., Vukmirović, P.: Superposition for full higher-order logic. In: Platzer, A., Sutcliffe, G. (eds.) CADE. LNCS, vol. 12699, pp. 396–412. Springer (2021)
5. Bentkamp, A., Blanchette, J., Tourret, S., Vukmirović, P., Waldmann, U.: Superposition with λ das. *J. Autom. Reason.* 65(7), 893–940 (2021)
6. Bentkamp, A., Blanchette, J.C., Cruanes, S., Waldmann, U.: Superposition for λ -free higher-order logic. In: Galmiche, D., Schulz, S., Sebastiani, R. (eds.) IJCAR. LNCS, vol. 10900, pp. 28–46. Springer (2018)
7. Bhayat, A., Reger, G.: Restricted combinatory unification. In: Fontaine, P. (ed.) CADE. LNCS, vol. 11716, pp. 74–93. Springer (2019)
8. Bhayat, A., Reger, G.: A combinator-based superposition calculus for higher-order logic. In: Peltier, N., Sofronie-Stokkermans, V. (eds.) IJCAR (1). LNCS, vol. 12166, pp. 278–296. Springer (2020)
9. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. *J. Formaliz. Reason.* 9(1), 101–148 (2016)
10. Brown, C.E.: Satallax: An automatic higher-order prover. In: Gramlich, B., Miller, D., Sattler, U. (eds.) IJCAR. LNCS, vol. 7364, pp. 111–117. Springer (2012)
11. Brown, C.E., Kaliszyk, C.: Lash 1.0 (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) IJCAR 2022. LNCS, vol. 13385, pp. 350–358. Springer (2022)
12. Cervesato, I., Pfenning, F.: A linear spine calculus. *J. Log. Comput.* 13(5), 639–688 (2003)
13. Charguéraud, A.: The locally nameless representation. *J. Autom. Reason.* 49(3), 363–408 (2012)
14. Cruanes, S.: Extending Superposition with Integer Arithmetic, Structural Induction, and Beyond. PhD thesis, École Polytechnique (2015)
15. Desharnais, M., Vukmirović, P., Blanchette, J., Wenzel, M.: Seventeen provers under the hammer. In: Andronick, J., de Moura, L. (eds.) ITP. LIPIcs, vol. 237, pp. 8:1–8:18. Schloss Dagstuhl (2022)
16. Gu, R., Shao, Z., Chen, H., Wu, X.N., Kim, J., Sjöberg, V., Costanzo, D.: CertiKOS: An extensible architecture for building certified concurrent OS kernels. In: Keeton, K., Roscoe, T. (eds.) OSDI. pp. 653–669. USENIX Association (2016)
17. Hales, T.C., Adams, M., Bauer, G., Dang, D.T., Harrison, J., Hoang, T.L., Kaliszyk, C., Magron, V., McLaughlin, S., Nguyen, T.T., Nguyen, T.Q., Nipkow, T., Obua, S., Pleso, J., Rute, J., Solovyev, A., Ta, A.H.T., Tran, T.N., Trieu, D.T., Urban, J., Vu, K.K., Zumkeller, R.: A formal proof of the Kepler conjecture. CoRR abs/1501.02155 (2015)

18. Hoder, K., Voronkov, A.: Sine qua non for large theory reasoning. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE. LNCS, vol. 6803, pp. 299–314. Springer (2011)
19. Hughes, R.J.M.: Super combinators: A new implementation method for applicative languages. In: Park, D.M.R., Friedman, D.P., Wise, D.S., Jr., G.L.S. (eds.) LFP. pp. 1–10. ACM (1982)
20. Kamareddine, F.: Reviewing the classical and the de Bruijn notation for λ -calculus and pure type systems. *J. Log. Comput.* 11(3), 363–394 (2001)
21. Kern, C., Greenstreet, M.R.: Formal verification in hardware design: A survey. *ACM Trans. Design Autom. Electr. Syst.* 4(2), 123–193 (1999)
22. Klein, G., Andronick, J., Elphinstone, K., Heiser, G., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., Winwood, S.: seL4: Formal verification of an operating-system kernel. *Commun. ACM* 53(6), 107–115 (2010)
23. Kotelnikov, E., Kovács, L., Suda, M., Voronkov, A.: A clausal normal form translation for FOOL. In: Benz Müller, C., Sutcliffe, G., Rojas, R. (eds.) GCAI. EPIc, vol. 41, pp. 53–71. EasyChair (2016)
24. Leroy, X.: Formal verification of a realistic compiler. *Commun. ACM* 52(7), 107–115 (2009)
25. Löchner, B., Schulz, S.: An evaluation of shared rewriting. In: de Nivelles, H., Schulz, S. (eds.) IWIL. pp. 33–48. Max-Planck-Institut für Informatik (2001)
26. McCune, W.: Experiments with discrimination-tree indexing and path indexing for term retrieval. *J. Autom. Reason.* 9(2), 147–167 (1992)
27. Nipkow, T.: Functional unification of higher-order patterns. In: Best, E. (ed.) LICS. pp. 64–74. IEEE Computer Society (1993)
28. Paulson, L.C., Blanchette, J.C.: Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In: Sutcliffe, G., Schulz, S., Ternovska, E. (eds.) IWIL. EPIc, vol. 2, pp. 1–11. EasyChair (2012)
29. Schulz, S.: E—a brainiac theorem prover. *AI Commun.* 15(2-3), 111–126 (2002)
30. Schulz, S.: Fingerprint indexing for paramodulation and rewriting. In: Gramlich, B., Miller, D., Sattler, U. (eds.) IJCAR. LNCS, vol. 7364, pp. 477–483. Springer (2012)
31. Schulz, S.: Simple and efficient clause subsumption with feature vector indexing. In: Bonacina, M.P., Stickel, M.E. (eds.) Automated Reasoning and Mathematics—Essays in Memory of William W. McCune. LNCS, vol. 7788, pp. 45–67. Springer (2013)
32. Schulz, S., Cruanes, S., Vukmirović, P.: Faster, higher, stronger: E 2.3. In: Fontaine, P. (ed.) CADE. LNCS, vol. 11716, pp. 495–507. Springer (2019)
33. Steen, A.: Extensional paramodulation for higher-order logic and its effective implementation leo-iii. *Künstliche Intell.* 34(1), 105–108 (2020)
34. Steen, A., Benz Müller, C.: There is no best β -normalization strategy for higher-order reasoners. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) LPAR-20 2015. LNCS, vol. 9450, pp. 329–339. Springer (2015)
35. Steen, A., Benz Müller, C.: Extensional higher-order paramodulation in Leo-III. *J. Autom. Reason.* 65(6), 775–807 (2021)
36. Stump, A., Sutcliffe, G., Tinelli, C.: StarExec: A cross-community infrastructure for logic solving. In: Demri, S., Kapur, D., Weidenbach, C. (eds.) IJCAR. LNCS, vol. 8562, pp. 367–373. Springer (2014)
37. Sultana, N., Blanchette, J.C., Paulson, L.C.: LEO-II and Satallax on the Sledgehammer test bench. *J. Applied Logic* 11(1), 91–102 (2013)
38. Sutcliffe, G.: The TPTP problem library and associated infrastructure—from CNF to TH0, TPTP v6.4.0. *J. Autom. Reason.* 59(4), 483–502 (2017)

39. Sutcliffe, G.: The 10th IJCAR automated theorem proving system competition—CASC-J10. *AI Commun.* 34(2), 163–177 (2021)
40. Vukmirović, P., Bentkamp, A., Blanchette, J., Cruanes, S., Nummelin, V., Tourret, S.: Making higher-order superposition work. In: Platzter, A., Sutcliffe, G. (eds.) *CADE. LNCS*, vol. 12699, pp. 415–432. Springer (2021)
41. Vukmirović, P., Bentkamp, A., Nummelin, V.: Efficient full higher-order unification. In: Ariola, Z.M. (ed.) *FSCD. LIPIcs*, vol. 167, pp. 5:1–5:17. Schloss Dagstuhl (2020)
42. Vukmirović, P., Blanchette, J.C., Cruanes, S., Schulz, S.: Extending a brainiac prover to lambda-free higher-order logic. In: Vojnar, T., Zhang, L. (eds.) *TACAS. LNCS*, vol. 11427, pp. 192–210. Springer (2019)
43. Vukmirović, P., Nummelin, V.: Boolean reasoning in a higher-order superposition prover. In: Fontaine, P., Korovin, K., Kotsireas, I.S., Rümmer, P., Tourret, S. (eds.) *PAAR+SC². CEUR Workshop Proceedings*, vol. 2752, pp. 148–166. CEUR-WS.org (2020)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

